complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + allmatch() rule checks

filter() null checks + allmatch() rule checks

filter() comparisons + allmatch() rule checks

filter() method calls returning boolean + allmatch() rule checks

distinct() rely on equals() / hashcode() + allmatch() rule checks

distinct() immutable objects + allmatch() rule checks

map() transform objects + allmatch() rule checks

map() create new objects + allmatch() rule checks

map() call pure functions + allmatch() rule checks

flatmap() return stream + allmatch() rule checks

flatmap() split data + allmatch() rule checks

flatmap() expand collections + allmatch() rule checks

sorted() natural ordering + allmatch() rule checks

sorted() custom comparator + allmatch() rule checks

sorted() multi-field compare + allmatch() rule checks

peek() logging + allmatch() rule checks

peek() debugging + allmatch() rule checks

peek() metrics + allmatch() rule checks

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = allMatch()**
- Purpose = **rule checks** (business / validation rules)
- **Short-circuiting behavior** is intentional
- Java **8+**
- No collectors, no reduce, no side-effect mutation (except peek demos)

---

**1 filter() boolean checks + allMatch() — Rule checks**

```
import java.util.*;

public class FilterBooleanAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 4, 6, 8);

      boolean rulePassed = true;

      for (Integer n : list) {
         if (n > 0) {                                    // filter
            if (n % 2 != 0) {                            // allMatch condition
               rulePassed = false;
               break;
            }
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**2 filter() null checks + allMatch() — Rule checks**

```
import java.util.*;

public class FilterNullAllMatchRule {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");

      boolean rulePassed = true;

      for (String name : list) {
         if (name != null) {
            if (name.length() < 4) {
               rulePassed = false;
               break;
            }
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**3 filter() comparisons + allMatch() — Rule checks**

```
import java.util.*;

public class FilterComparisonAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(25, 30, 40);

      boolean rulePassed = true;

      for (Integer age : list) {
         if (age >= 18) {
            if (age > 60) {
               rulePassed = false;
               break;
            }
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**4 filter() method calls returning boolean + allMatch() — Rule checks**

```
import java.util.*;

public class FilterMethodAllMatchRule {

   static class User {
      boolean verified;
      User(boolean verified) { this.verified = verified; }
      boolean isVerified() { return verified; }
   }

   public static void main(String[] args) {

      List<User> list = Arrays.asList(
            new User(true),
            new User(true)
      );

      boolean rulePassed = true;

      for (User u : list) {
         if (u.isVerified()) {
            if (!u.isVerified()) {
               rulePassed = false;
               break;
            }
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**5 distinct() rely on equals() / hashCode() + allMatch() — Rule checks**

```
import java.util.*;

public class DistinctEqualsAllMatchRule {

   static class Person {
      String email;

      Person(String email) { this.email = email; }

      public boolean equals(Object o) {
         if (this == o) return true;
         if (!(o instanceof Person)) return false;
         Person p = (Person) o;
         return email.equals(p.email);
      }

      public int hashCode() {
         return email.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> list = Arrays.asList(
            new Person("a@mail.com"),
            new Person("a@mail.com"),
            new Person("b@mail.com")
      );

      Set<Person> unique = new HashSet<Person>(list);

      boolean rulePassed = true;

      for (Person p : unique) {
         if (!p.email.contains("@")) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**6 distinct() immutable objects + allMatch() — Rule checks**

```
import java.util.*;

public class DistinctImmutableAllMatchRule {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java", "Java", "Python");
      Set<String> unique = new HashSet<String>(list);

      boolean rulePassed = true;

      for (String lang : unique) {
         if (lang.length() < 4) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**7 map() transform objects + allMatch() — Rule checks**

```
import java.util.*;

public class MapTransformAllMatchRule {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("anurag", "amit");

      boolean rulePassed = true;

      for (String s : list) {
         String upper = s.toUpperCase();
         if (!upper.equals(upper.toUpperCase())) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**8 map() create new objects + allMatch() — Rule checks**

```
import java.util.*;

public class MapCreateObjectAllMatchRule {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");

      boolean rulePassed = true;

      for (String name : list) {
         UserDTO dto = new UserDTO(name);
         if (dto.name.length() < 4) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**9 map() call pure functions + allMatch() — Rule checks**

```
import java.util.*;

public class MapPureFunctionAllMatchRule {

   static int normalize(int n) { return n * 10; }

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      boolean rulePassed = true;

      for (Integer n : list) {
         int value = normalize(n);
         if (value < 10) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**10 flatMap() return Stream + allMatch() — Rule checks**

```
import java.util.*;

public class FlatMapStreamAllMatchRule {
   public static void main(String[] args) {

      List<List<Integer>> list = new ArrayList<List<Integer>>();
      list.add(Arrays.asList(1,2));
      list.add(Arrays.asList(3,4));

      boolean rulePassed = true;

      for (List<Integer> inner : list) {
         for (Integer n : inner) {
            if (n <= 0) {
               rulePassed = false;
               break;
            }
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**11 flatMap() split data + allMatch() — Rule checks**

```
import java.util.*;

public class FlatMapSplitAllMatchRule {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java Stream", "Flat Map");

      boolean rulePassed = true;

      for (String s : list) {
         String[] words = s.split(" ");
         for (String word : words) {
            if (word.length() < 3) {
               rulePassed = false;
               break;
            }
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**12 flatMap() expand collections + allMatch() — Rule checks**

```
import java.util.*;

public class FlatMapExpandAllMatchRule {
   public static void main(String[] args) {

      Map<String, List<String>> map = new HashMap<String, List<String>>();
      map.put("A", Arrays.asList("1", "2"));
      map.put("B", Arrays.asList("3"));

      boolean rulePassed = true;

      for (List<String> values : map.values()) {
         for (String v : values) {
            if (v.length() == 0) {
               rulePassed = false;
               break;
            }
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**13 sorted() natural ordering + allMatch() — Rule checks**

```
import java.util.*;

public class SortedNaturalAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = new ArrayList<Integer>(Arrays.asList(1,2,3));
      Collections.sort(list);

      boolean rulePassed = true;

      for (Integer n : list) {
         if (n < 1) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**14 sorted() custom comparator + allMatch() — Rule checks**

```
import java.util.*;

public class SortedCustomAllMatchRule {
   public static void main(String[] args) {

      List<String> list = new ArrayList<String>(
            Arrays.asList("Anurag", "Ravi", "Amit")
      );

      Collections.sort(list, new Comparator<String>() {
         public int compare(String a, String b) {
            return b.compareTo(a);
         }
      });

      boolean rulePassed = true;

      for (String name : list) {
         if (name.length() < 4) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**15 sorted() multi-field compare + allMatch() — Rule checks**

```
import java.util.*;

public class SortedMultiFieldAllMatchRule {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> list = new ArrayList<Employee>();
      list.add(new Employee("Anurag", 30));
      list.add(new Employee("Anurag", 25));
      list.add(new Employee("Ravi", 35));

      Collections.sort(list, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {
            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) return nameCompare;
            return e1.age - e2.age;
         }
      });

      boolean rulePassed = true;

      for (Employee e : list) {
         if (e.age < 18) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

**16 peek() logging + allMatch() — Rule checks**

```
import java.util.*;

public class PeekLoggingAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1,2,3);

      boolean rulePassed = true;

      for (Integer n : list) {
         System.out.println("Checking: " + n);
         if (n <= 0) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**17 peek() debugging + allMatch() — Rule checks**

```
import java.util.*;

public class PeekDebugAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10,20,30);

      boolean rulePassed = true;

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug breakpoint");
         }
         if (n < 10) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

**18 peek() metrics + allMatch() — Rule checks**

```
import java.util.*;

public class PeekMetricsAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5,10,15);

      int processed = 0;
      boolean rulePassed = true;

      for (Integer n : list) {
         processed++;
         if (n <= 0) {
            rulePassed = false;
            break;
         }
      }

      System.out.println("Rule passed: " + rulePassed);
      System.out.println("Processed elements: " + processed);
   }
}
```

---